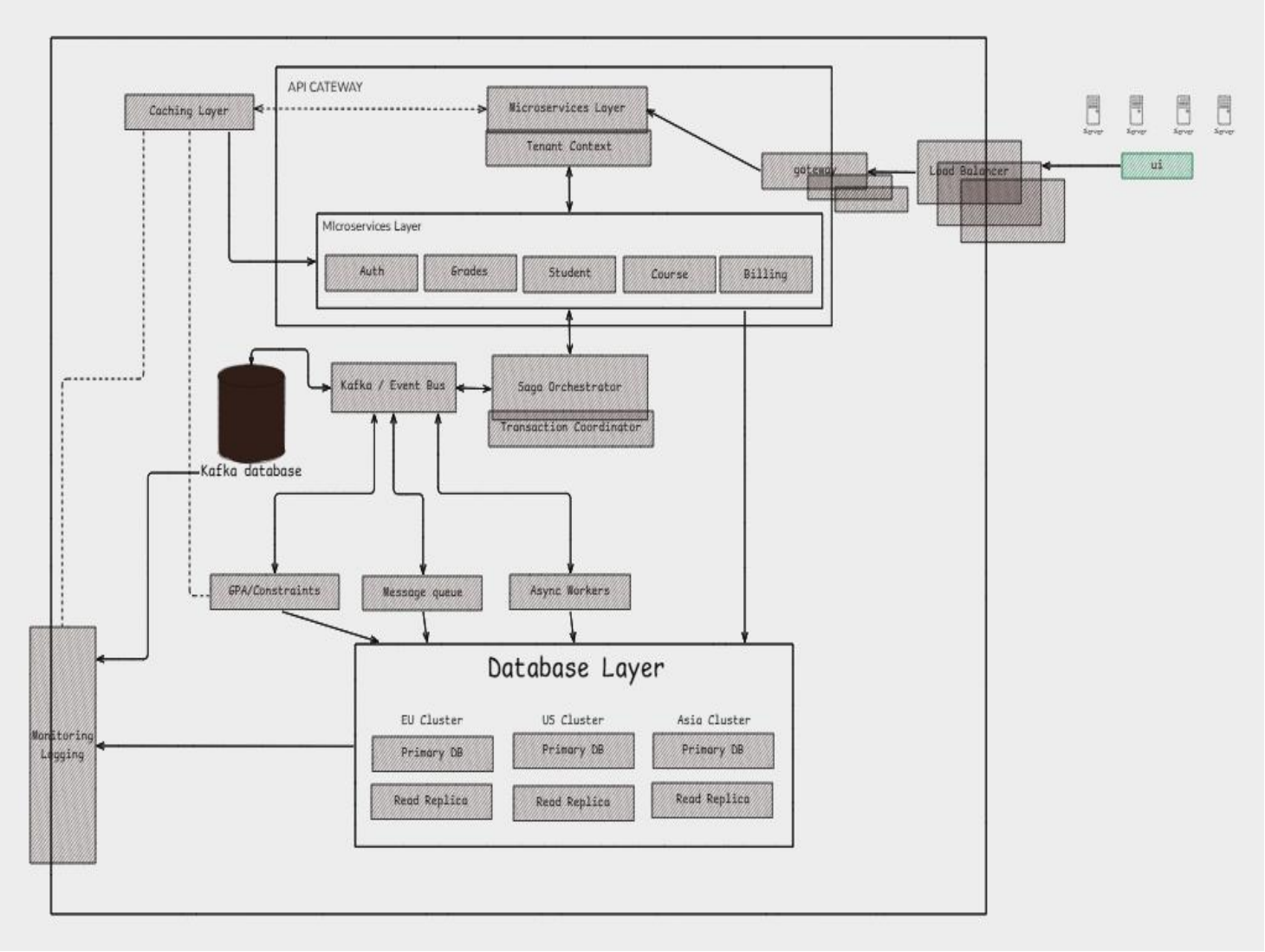
The objective is to design a global student portal similar in functionality to a university-specific academic system, but scalable and accessible to multiple universities worldwide. The system must support millions of users, operate across diverse geographic regions, and maintain strict consistency of academic records. The system must support multi-tenant access for multiple universities; scale horizontally under peak load; ensure high availability (>99.9%); maintain strict consistency for grades, GPA, transcripts, and enrollment records; provide secure tenant-level isolation; tolerate infrastructure failures across regions.

- The functional Requirements:

The platform must provide user authentication and authorization; course enrollment; grade management; transcript access; schedule viewing; billing and payment tracking; notification services; administrative dashboards per university.

- The Non-functional Requirements:

Horizontal scalability; Strong consistency for academic data; Fault tolerance; Multi-region deployment; Multi-tenancy isolation; Observability and monitoring; Secure access control (RBAC).



Multi-Tenancy Model: Each university operates as a logical tenant.

A hybrid strategy is adopted: Small and mid-size institutions share database clusters; Large institutions may use dedicated database instances.

In shared database environments, tenant isolation is enforced using Row-Level Security (RLS). All tables include a **tenant_id** attribute, and database-level policies ensure that every query is automatically restricted to the authenticated tenant context. This guarantees strict data isolation at the SQL engine level, preventing cross-tenant data exposure even in case of application-layer misconfiguration.

Scalability and Capacity Estimation:

Assumptions: 500 universities; 20000 students per university; Total users = 10^6

If 20% are daily active (2 million) and 10% are active concurrently during peak hours (200,000 concurrent users), with 5 requests per minute per user:

2*10^5 * 5 = 10^6 requests per minute = 16600 requests per second (RPS)

If one stateless application instance supports 1000 RPS: 16600 / 1000 = approximately 17 instances

For redundancy and headroom: 25–30 instances behind load balancers.

Auto-scaling policy: scale-out at CPU > 70%; scale-in at CPU < 40%.

Database Architecture and Sharding: Traffic distribution: 60% reads and 40% writes

Estimated load:

approximately 10,000 read RPS
approximately 6,600 write RPS

Architecture: Primary database node for write operations; Multiple read replicas for scaling read queries; Sharding by tenant_id; Regional database clusters (e.g., EU, Asia, US).

This reduces cross-region latency and improves performance.

Distributed Transactions: Saga Pattern:
Because the system uses microservices, traditional ACID transactions across services are not feasible.

For example: Enrollment Service must reserve a seat; Billing Service must process payment.

To maintain consistency, the system applies the **Saga pattern** using orchestration.Sequence: Enrollment Service initiates seat reservation, billing Service processes payment and if payment fails, a compensating transaction cancels enrollment. This approach ensures distributed consistency without relying on two-phase commit, preserving scalability while maintaining business integrity.

Event-Driven Architecture: Message queues are used for: notifications, GPA recalculation, transcript generation, audit logging, billing events. Producer to Message Queue to Consumer. This design isolates heavy processes from synchronous request flows and stabilizes system behavior during enrollment spikes.

Consistency Model: CAP and PACELC:

In globally distributed systems, network partitions are inevitable.

Under CAP: Partition tolerance is mandatory. For academic records, consistency is prioritized over availability (CP-oriented design).

However, beyond CAP, system behavior aligns with the **PACELC principle**:

If a partition occurs-prioritize Consistency.

Else-balance Latency and Consistency.

Grades and enrollment require strong consistency.
Notifications and analytics may tolerate eventual consistency.

Time Synchronization and Ordering:
In a multi-region system, consistent time ordering is critical for: grade publication timestamps, audit logs, enrollment ordering.

The system uses:

1)NTP-based clock synchronization across nodes;

2)logical timestamps for event sequencing;

3)consistent ordering guarantees within message queues.

This prevents race conditions and ensures correct event ordering across regions.

Fault Tolerance:
Fault tolerance is achieved via: multiple stateless application instances, database replication, automatic failover, regional redundancy, health checks and circuit breakers, DNS-based traffic routing during region failure.

Application node failure does not affect availability. Database replica failure triggers automatic promotion. Regional outage redirects traffic to secondary clusters.

Security Architecture, security mechanisms include: JWT-based authentication, role-based access control, tenant-aware authorization, TLS encryption in transit, encryption at rest, Row-Level Security enforcement, audit logging for compliance.

In conclusion, the proposed architecture delivers a globally distributed, multi-tenant student portal capable of serving millions of users across hundreds of universities. It ensures strict academic data consistency while maintaining horizontal scalability and fault tolerance. Through the integration of: hybrid multi-tenancy with Row-Level Security, Saga-based distributed transaction management, CP-oriented consistency model, PACELC-informed latency trade-offs and synchronized time mechanisms, the system achieves enterprise-grade reliability, scalability, and data integrity suitable for worldwide academic deployment.